Modul 6

Prozedurale SQL-Erweiterungen

Josef Altmann



Der vorliegende Foliensatz basiert auf:
Saake, G., Sattler, K.-U., Heuer, A.: *Datenbanken: Konzepte und Sprachen*, MITP, 6. Auflage (2018)
Loney, K: *Oracle Database 11g – Die umfassende Referenz*, Hanser Verlag (2009)
PL/SQL Language Reference, docs.oracle.com/cd/B28359_01/appdev.111/b28370.pdf

Überblick

- Einführung und Motivation
- Deklarationsteil
- Ausführungsteil
- Cursorkonzept
- Gespeicherte Programme
- Pakete
- Ausnahmebehandlung
- Rechte

© 2025

Einführung
Fallbeispiel

Mitarbeiter

SVNR
...

- Jährliche elten folgende
  - Gehalt Gehalt
  - Gehalts sen, also un

Deine Lohn- und Gehaltserhöhung

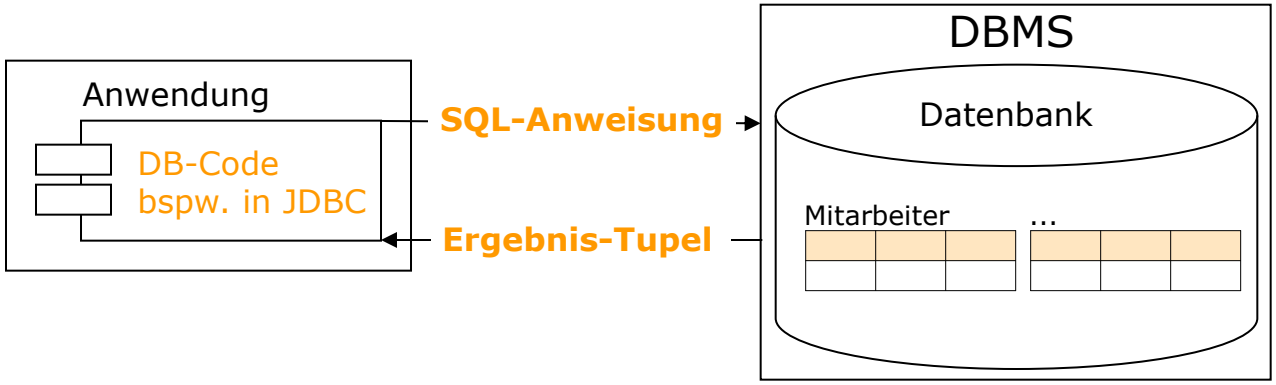
Grundlohn, -gehalt	Prozentuelle Erhöhung	Summe	plus Fixbetrag (jedes Monat)	Erhöhung in Prozent (Nachhaltig und dauerhaft!)
	5,4%		75,00 €	
2.100,00 €	113,40 €	2.213,40 €	2.288,40 €	8,9%
2.200,00 €	118,80 €	2.318,80 €	2.393,80 €	8,8%
2.300,00 €	124,20 €	2.424,20 €	2.499,20 €	8,7%
2.400,00 €	129,60 €	2.529,60 €	2.604,60 €	8,5%
2.500,00 €	135,00 €	2.635,00 €	2.710,00 €	8,4%
2.600,00 €	140,40 €	2.740,40 €	2.815,40 €	8,3%
2.700,00 €	145,80 €	2.845,80 €	2.920,80 €	8,2%
2.800,00 €	151,20 €	2.951,20 €	3.026,20 €	8,1%
2.900,00 €	156,60 €	3.056,60 €	3.131,60 €	8,0%
3.000,00 €	162,00 €	3.162,00 €	3.237,00 €	7,9%
3.100,00 €	167,40 €	3.267,40 €	3.342,40 €	7,8%
3.200,00 €	172,80 €	3.372,80 €	3.447,80 €	7,7%
3.300,00 €	178,20 €	3.478,20 €	3.553,20 €	7,7%
3.400,00 €	183,60 €	3.583,60 €	3.658,60 €	7,6%
3.500,00 €	189,00 €	3.689,00 €	3.764,00 €	7,5%
3.600,00 €	194,40 €	3.794,40 €	3.869,40 €	7,5%
3.700,00 €	199,80 €	3.899,80 €	3.974,80 €	7,4%
3.800,00 €	205,20 €	4.005,20 €	4.080,20 €	7,4%
3.900,00 €	210,60 €	4.110,60 €	4.185,60 €	7,3%
4.000,00 €	216,00 €	4.216,00 €	4.291,00 €	7,3%
4.100,00 €	221,40 €	4.321,40 €	4.396,40 €	7,2%
4.200,00 €	226,80 €	4.426,80 €	4.501,80 €	7,2%
4.300,00 €	232,20 €	4.532,20 €	4.607,20 €	7,1%
4.400,00 €	237,60 €	4.637,60 €	4.712,60 €	7,1%
4.500,00 €	243,00 €	4.743,00 €	4.818,00 €	7,1%
4.600,00 €	248,40 €	4.848,40 €	4.923,40 €	7,0%
4.700,00 €	253,80 €	4.953,80 €	5.028,80 €	7,0%
4.800,00 €	259,20 €	5.059,20 €	5.134,20 €	7,0%
4.900,00 €	264,60 €	5.164,60 €	5.239,60 €	6,9%
5.000,00 €	270,00 €	5.270,00 €	5.345,00 €	6,9%

Tabelle gilt für Vollzeitbeschäftigung

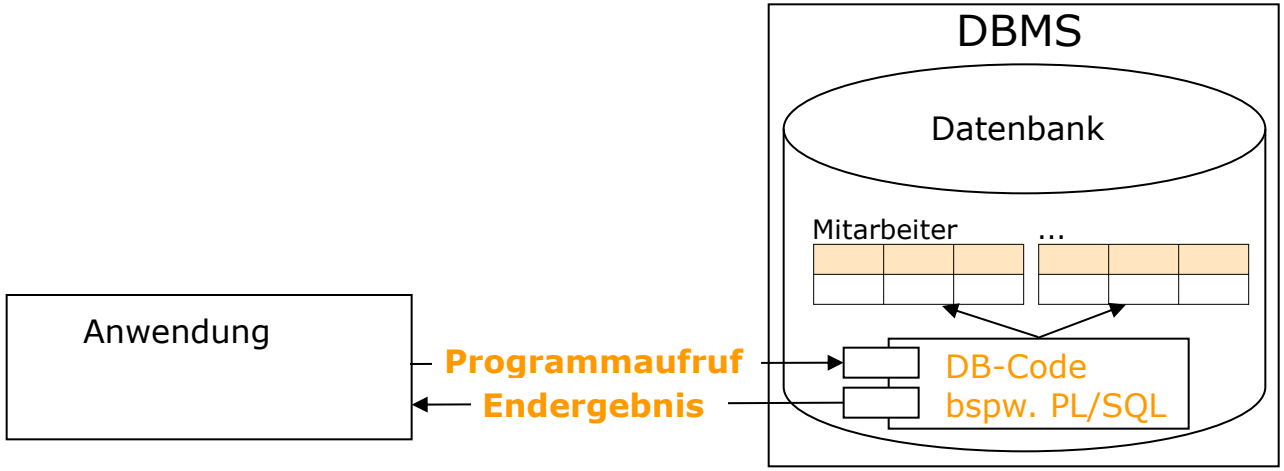
© 2025

Einführung
Fallbeispiel: Lösungsmöglichkeiten

- Datenbankmanipulationslogik in der Anwendung



- Datenbankmanipulationslogik in der Datenbank



Einführung
Anwendungsentwicklung

Prozedurale SQL-Erweiterung

Bsp.: JDBC

```
import java.sql.*;
public class MySelect {
   Connection con = null; Statement stmt = null; ResultSet rs = null;
   public static void main (...) {
      try { con = DriverManager.getConnection(
                  "jdbc:oracle:thin:@localhost:1521:orcl",
                  USERNAME, PASSWORD);
        stmt = con.createStatement();
        rs = stmt.executeQuery("SELECT nr, ... FROM TEST);
        while (rs.next()) {
           System.out.println("Nr:   "+rs.getString(1));
           ...
        }
        ...
```

Kopplung mit Datenbank

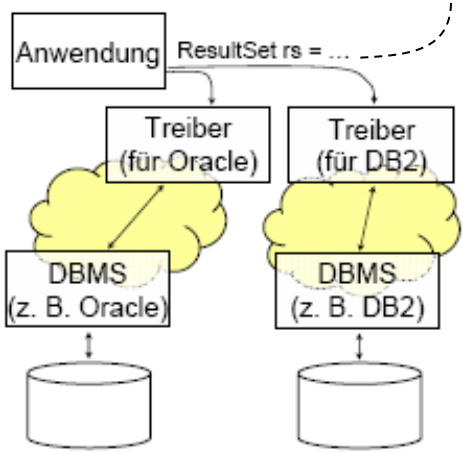
Eingebettetes statisches SQL
Bsp.: SQLJ

```
public void employee(String empNo) {
   String lastName = null; ...
   // Get the connection
   con = DriverManager.getConnection(url);
   //Lookup the employee
   #sql {SELECT LASTNAME INTO :lastName
         FROM EMPLOYEE WHERE EMPNO=:empNo};
   System.out.println (lastName); ...
```

enge Kopplung mit DB-Server

PL/SQL, Transact-SQL, PL/pgSQL etc.
Anwendungslogik wird in die Datenbank verlagert
⇨ gespeicherte Prozeduren und Funktionen

Dynamisches SQL
CLI



Bsp.: Embedded C

```
exec sql include sqlca;
main () {
   exec sql begin declare section;
      char X[3]; int GSum;
   exec sql end declare section;
   exec sql connect to dbname;
   exec sql    insert into Pers (Pnr, Name)
            values (4711, 'Ernie');
   ...
   exec sql    select sum (Gehalt) into :GSum
               from Pers where Anr = :X;
   printf ("Gehaltssumme: %d\n" , GSum)
   exec sql commit work;
   exec sql disconnect;
}...
```

Zugriff „von irgendwo"

eng

statisch
Embedded SQL, SQLJ etc.

dynamisch

Erzeugung SQL-Code

generisch

Call Level Interface (CLI)
z.B. ODBC, JDBC, JPA, ADO.NET, LINQ

Einbettung in Programmiersprache

Einführung

Schwächen von SQL – Vorteile prozeduraler SQL-Erweiterungen

- **Interaktives SQL** für viele DB-Applikationen **nicht ausreichend**
  - nur sequentielle Abarbeitung von SQL-Anweisungen
  - weder Schleifen noch bedingte Verzweigung
  - keine Berechnungsvollständigkeit
  - keine Modularisierung
- primitive Lösung **CLI** und **Embedded SQL** suboptimal
  - ständiger Wechsel zwischen Anwendung und DBS
  - keine Anweisungsübergreifende Optimierung
- **Prozedurale Erweiterungen** von SQL kombiniert die
  - Stärken einer **deklarativen Abfragesprache** mit den
  - Stärken einer **prozeduralen Programmiersprache** (Ablaufstruktur bzw. Anwendungslogik)

© 2025

Einführung

Vorteile von gespeicherten Prozeduren/Funktionen

- Vorübersetzte Ausführungspläne
  - wiederverwendbar
  - optimiert
- Performance-Steigerung: Datenbank-Zugriffe der Anwendungen werden reduziert
- Zentrale „Geschäftslogik“
  - nutzbar von verschiedenen Anwendungsprogrammen
  - höherer Isolationsgrad der Anwendung von der Datenbank (Schichtenarchitektur)
  - Rechtevergabe für Prozeduren/Funktionen
- Nachvollziehbarkeit von Datenmanipulationen im Fehlerfall
- keine „externen“ Transaktionen
- Zur Integritätssicherung: Aktionsteil von Triggern

© 2025

Einführung

Nachteile von gespeicherten Prozeduren/Funktionen

- DBMS wird stärker belastet
- kompliziertere Fehlersuche (Debugging)
- kaum Benutzerinteraktion möglich
- zusätzlich „hybride“ Sprache notwendig

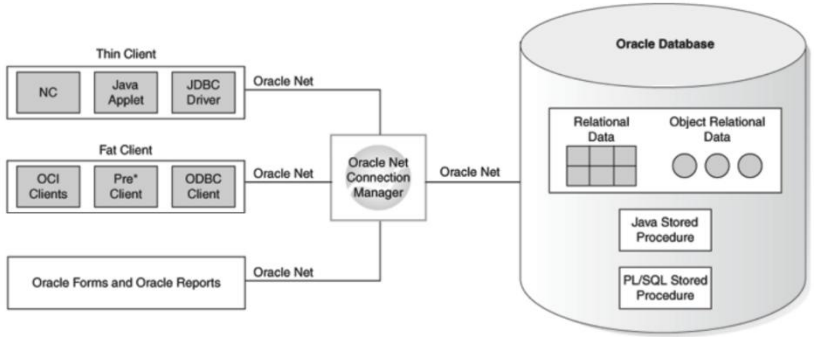
© 2025

Einführung
Prozedurale Erweiterungen im SQL-Standard

- Prozedural erweitertes SQL für Stored Procedures im SQL/PSM (Persistent Stored Module)- und im SQL-Invoked Routines-Standard definiert:
  - gespeicherte Module bestehend aus Prozeduren und Funktionen
  - Prozeduren und Funktionen können auch einzeln gespeichert werden
  - syntaktische Konstrukte für Schleifen, Bedingungen etc.
  - Ausnahmebehandlung
  - Einbindung externer Routinen (implementiert in C, Java, C#, . . .)
  - …

Einführung
Prozedurale Erweiterungen in DBMS-Produkten

- Entsprechende Produkte:
  - PostgreSQL: PL/pgSQL, PL/Python, PL/Perl, PL/Java
  - Informix: SPL
  - IBM DB2: SQL/PSM (Persistent Stored Modules)
  - MS SQL Server, Sybase: Transact-SQL, .NET Framework Sprachen
  - MySQL: SQL/PSM (Persistent Stored Modules); ab Version 5
  - Oracle: PL/SQL (Procedural Language/Structured Query Language), Java
  - …
- nicht vollständig konform zum SQL-Standard



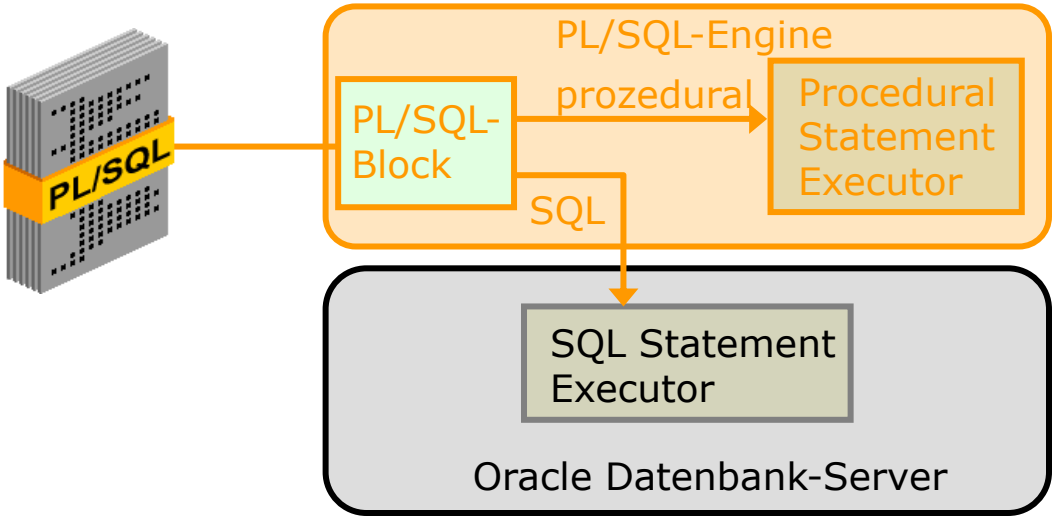
Einführung

ORACLE: PL/SQL (**P**rocedural **L**anguage/**S**tructured **Q**uery **L**anguage)

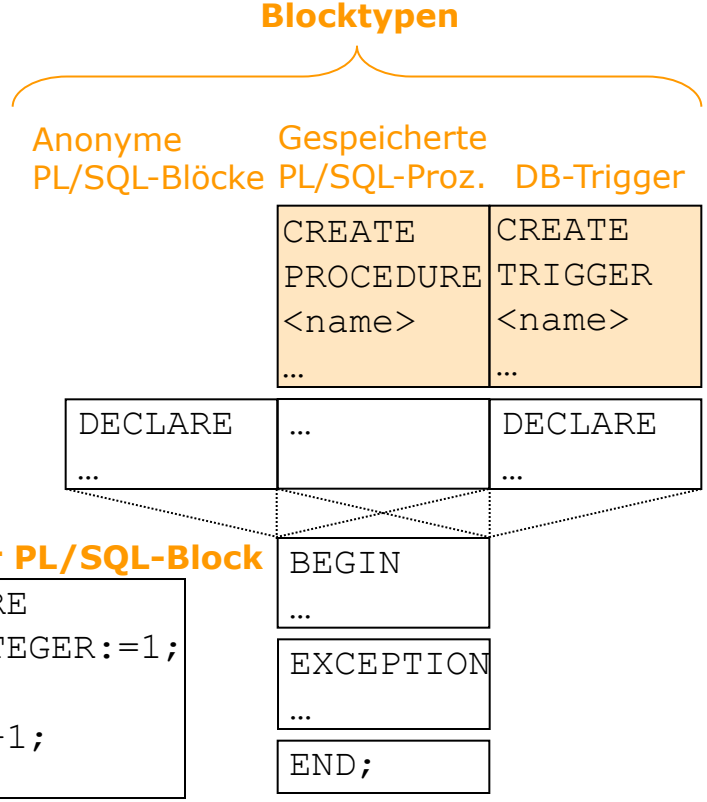
PL/SQL = Integration von prozeduralen Konstrukten mit SQL



- Ausführung komplexer DB-orientierter Operationen innerhalb des DB-Servers (PL/SQL-Umgebung)



Einführung
Sprachkonstrukte



- **Blockorientierte Sprache** mit **statischer Typprüfung**

- Bestandteile eines Blocks:
  - **Deklarationsteil** (optional)
    - Variablen/Konstanten
    - explizite Cursor
    - benutzerdefinierte Ausnahmen
  - **Ausführungsteil** (erforderlich)
    - SQL-Anweisungen
    - PL/SQL-Anweisungen
  - **Fehler- und Ausnahmebehandlungsteil** (optional)
    - Bearbeitung von Fehlersituationen durch „Exception Handler“

- Blöcke lassen sich schachteln, bilden Gültigkeitsbereiche

Überblick

- Einführung und Motivation
- Deklarationsteil
- Ausführungsteil
- Cursorkonzept
- Gespeicherte Programme
- Pakete
- Ausnahmebehandlung
- Rechte

© 2025

Deklarationsteil
Variablen und Konstanten

- Deklaration von
  - Name
  - Datentyp (explizit bzw. implizit)
  - Vorbelegung (bei Variablen optional)
- Variablendeklaration

<Variable> <Datentyp> [[NOT NULL] (:=|DEFAULT) <Ausdruck>];

```
name      VARCHAR2(50);
ok        BOOLEAN DEFAULT FALSE;
saldo     NUMBER(5) NOT NULL := 0;
```

nur in PL/SQL
nicht in SQL

- Konstantendeklaration

<Konstante> CONSTANT <Datentyp> (:=|DEFAULT) <Ausdruck>;

```
wert      CONSTANT NUMBER(8) DEFAULT 100;
pi        CONSTANT REAL := 3.14159;
```

Deklarationsteil

Implizite Datentypdefinition

Mitarbeiter

Vorname	Nachname	Geburtsdatum
...	...	...

- Typkonstruktoren: **%TYPE**, **%ROWTYPE**
  - Unabhängigkeit von Schemamodifikationen
  - Erzeugt „passenden" Typ, abhängig vom DB-Schema
    - aber ohne NOT NULL Constraint
- Deklaration über bereits definierten Datentyp: **%TYPE**
  - eines Attributs einer Tabelle/Sicht

```
name    mitarbeiter.nachname%TYPE;
```

  - einer bereits deklarierten Variable/Konstante

```
name2    name%TYPE;
```

- Deklaration über Tupel/Record-Struktur: **%ROWTYPE**
  - einer Tabelle/Sicht

```
datensatz_mitarbeiter    mitarbeiter%ROWTYPE;
```

  - eines bereits deklarierten Cursors

```
CURSOR mitarbeiter_cursor IS SELECT ... FROM ...;
datensatz_mitarbeiter2    mitarbeiter_cursor%ROWTYPE;
```

© 2025

Deklarationsteil
Komplexe Datentypen Überblick

- **PL/SQL-Record**

```
TYPE <RecordTyp> IS RECORD (<Vardekl> {, <Vardekl>});
```

- **PL/SQL-Collections**
  - PL/SQL-Nested Table

```
TYPE <TabellenTyp> IS TABLE OF <Domäne> [NOT NULL];
```

  - PL/SQL Associative Array

```
TYPE <TabellenTyp> IS TABLE OF <Domäne> [NOT NULL]
                   INDEXED BY <Domäne>;
```

  - PL/SQL-VArray

```
TYPE <ArrayTyp> IS VARRAY (<Grösse>) OF <Domäne> [NOT NULL];
```

- Deklaration von Records und Collections in zwei Schritten
  - Deklaration des gewünschten Datentyps
  - Variablendeklaration mithilfe des zuvor deklarierten Datentyps

© 2025

Deklarationsteil

Komplexe Datentypen: **PL/SQL Record**

- besteht aus **beliebigen Elementen beliebiger Datentypen**
- **atomare** oder **zusammengesetzte Datentypen** als Komponenten

```
TYPE MITARBEITER_REC_TYP IS RECORD (
  mit_name VARCHAR2(30) NOT NULL := 'Huber',
  mit_geb DATE,
  mit_ort mitarbeiter.ort%TYPE,
  mit_abtnr NUMBER(8) := 1);
mitarbeiter_record MITARBEITER_REC_TYP;
```

- **Zugriff** auf einzelne Komponenten über **Pfade**

```
mitarbeiter_record.mit_geb := TO_DATE('01-feb-93');
```

- Zuweisung eines Record auf Record gleichen Typs erlaubt
- **Strenge Typisierung**: Zuweisung eines Record auf Record anderen Typs auch bei exakter Übereinstimmung aller Komponenten nicht möglich

© 2025

Deklarationsteil

Komplexe Datentypen: **PL/SQL Collections**

- eindimensionale Collections eines skalaren oder benutzerdefinierten Datentyps (Record)
  - **Tabellen & Associative Arrays**: ungeordnet
  - **VArrays**: geordnet u. fixe Größe

```
TYPE TEXT_TABLE_TYP   IS TABLE OF VARCHAR2(80);
TYPE KEY_VALUE_TYP    IS TABLE OF VARCHAR2(80)
                      INDEXED BY VARCHAR2(10);
TYPE TEXT_VARRAY_TYP  IS VARRAY(10) OF VARCHAR2(80);

text_tab   TEXT_TABLE_TYP;
text_assoc KEY_VALUE_TYP;
text_va    TEXT_VARRAY_TYP;
```

Deklarationsteil
Komplexe Datentypen: **PL/SQL Collections**

- Konstruktor-Aufruf

```
text_tab := TEXT_TABLE_TYP('Zwei', 'Wörter');
```

- vordefinierte PL/SQL-Funktionen: `<Collection>.<Funktion>`
  `EXISTS(i), FIRST, LAST, COUNT, NEXT(i), EXTEND(i), DELETE(i), ...`

```
anzahl := text_tab.COUNT();
```

- Zugriff auf einzelne Elemente über ganzzahligen Index

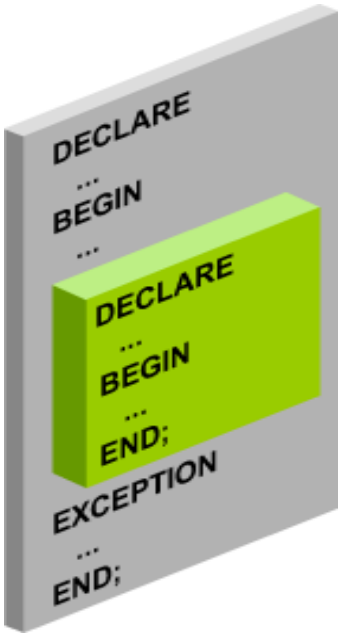
```
text_tab(1) := 'PL/SQL ist eine DB-Sprache von Oracle';
DBMS_OUTPUT.PUTLINE(text_tab(text_tab.NEXT(FIRST)))
```

- Elemente ohne Wert, existieren noch nicht
  - Lesezugrifffehler `NO_DATA_FOUND (ORA-01402)`
- können nicht direkt auf Gleichheit getestet werden

Überblick

- Einführung und Motivation
- Deklarationsteil
- Ausführungsteil
- Cursorkonzept
- Gespeicherte Programme
- Pakete
- Ausnahmebehandlung
- Rechte

Ausführungsteil
Verschachtelte Blöcke



- PL/SQL-Blöcke können verschachtelt werden
  - `BEGIN ... END` kann verschachtelte Blöcke enthalten.
  - Innerhalb eines Blocks dürfen sich nicht mehrere Variablen mit demselben Namen befinden.
  - Variablen mit demselben Namen können in verschiedenen (verschachtelten) Blöcken deklariert werden.

Ausführungsteil
Anweisungen

- Erlaubte Anweisungen
  - beliebige **DML-Anweisungen**

```
SELECT, INSERT, UPDATE, DELETE, ...
```

```
SELECT nr, name INTO m_nr, m_name FROM mitarbeiter
  WHERE ...;
```

```
UPDATE mitarbeiter SET gehalt = gehalt*1.1
  WHERE mitarbeiter_nr = 1 AND gehalt IS NOT NULL
  RETURNING name, gehalt INTO m_name, m_neues_gehalt;
```

  - Anweisungen für die **Transaktionskontrolle**
    - `COMMIT, ROLLBACK, SAVEPOINT, ...`
  - alle **SQL-Funktionen** inkl. aggregierender Funktionen
    - `LENGTH(), LOWER(), ...`
  - **SQL-Pseudospalten**
    - `CURRVAL, LEVEL, NEXTVAL, ROWID, ROWNUM`
  - **DDL-Anweisungen**: über dynamisches SQL oder DBMS_SQL-Paket
    - `EXECUTE IMMEDIATE 'CREATE TABLE Abteilung (ANr INTEGER)';`

© 2025

Ausführungsteil
Anweisungen - Beispiele

- Daten abfragen
  - **INTO**-Klausel ist obligatorisch
  - Abfragen dürfen nur eine Zeile zurückgeben

```
DECLARE
  fname VARCHAR2(25);
BEGIN
  SELECT first_name INTO fname
  FROM employees WHERE employee_id = 200;
END;
```

```
DECLARE
  emp_hiredate  employees.hire_date%TYPE;
  emp_salary    employees.salary%TYPE;
  emp_id    employees.employee_id%TYPE := 176;
BEGIN
  SELECT  hire_date, salary INTO emp_hiredate, emp_salary
  FROM    employees
  WHERE   employee_id = emp_id ;
END;
```

Ausführungsteil
Anweisungen - Beispiele

- Daten aktualisieren

```
DECLARE
  sal_increase     employees.salary%TYPE := 80;
BEGIN
  UPDATE    employees
  SET       salary = salary + sal_increase
  WHERE     job_id = 'ST_CLERK';
END;
```

- Daten löschen

```
DECLARE
  dept_id  employees.department_id%TYPE := 10;
BEGIN
  DELETE FROM  employees
  WHERE        department_id = dept_id;
END;
```

© 2025

Ausführungsteil
Anweisungen - Beispiele

- Daten aktualisieren mit **RETURNING**

```
DECLARE
  sal_increase employees.salary%TYPE;
BEGIN
  SELECT salary
    INTO sal_increase
    FROM employees
   WHERE employee_id = 100;

  DBMS_OUTPUT.PUT_LINE('Ergebnis: ' || sal_increase);

  UPDATE employees
     SET salary = salary * 1.1
   WHERE employee_id = 100
  RETURNING salary INTO sal_increase;

  DBMS_OUTPUT.PUT_LINE(sal_increase);
END;
```

© 2025

Ausführungsteil

Anweisungen – Auflösung von Variablennamen

- ACHTUNG: Datenbank-Spalten haben Vorrang bei gleichnamigen Variablen

```
DECLARE
  last_name VARCHAR2(25) := 'King';
BEGIN
  DELETE FROM employees
        WHERE last_name = last_name;
END;
```

- löscht alle Daten (`last_name = last_name`)

Ausführungsteil

Ablauf- und Kontrollstrukturen

Zuweisungen:

Zuweisungsausdruck	Beispiel
Konstanten	`var0 := 10;`
Variablen	`var1 := var2;`
Arithmetische Ausdrücke	`var3 := var1*faktor;`
SQL-Funktionen	`var4 := NVL(hire_date,SYSDATE);`
Gespeicherte Funktionen	`var5 := meine_funk(var1);`
Bedingungen	`var6 := (gehalt > 3500);`

Bedingte Verarbeitung:

```
IF datum < '26-mai-93' THEN
  INSERT INTO tabX VALUES (...);
ELSIF name LIKE '%er' THEN
  a := vor_name;
ELSIF name LIKE 'A%' THEN
  a := name;
ELSE
  abc(datum, name, vor_name, status);
END IF;
```

```
CASE <Ausdruck>
  WHEN <Ausdruck> THEN <Anweisungen>;
...
  [ELSE <Anweisungen>;]
END CASE;
```

```
CASE
  WHEN <Bedingung> THEN <Anweisungen>;
...
  [ELSE <Anweisungen>;]
END CASE;
```

© 2025

Ausführungsteil
CASE-Anweisung

- CASE-Ausdruck prüft Bedingung und **gibt Wert zurück**

```
DECLARE
  grade CHAR(1) := UPPER('&grade');
  appraisal VARCHAR(20);
BEGIN
  appraisal :=    CASE grade
                  WHEN 'A' THEN 'Excellent'
                  WHEN 'B' THEN 'Very Good'
                  ...
                  ELSE 'Invalid grade'
              END;
END;
```

Expliziter Ausdruck

```
DECLARE
  grade CHAR(1) := UPPER('&grade');
  appraisal VARCHAR(20);
BEGIN
  appraisal :=    CASE
                  WHEN grade = 'A' THEN 'Excellent'
                  WHEN grade IN ('B', 'C') THEN 'Very Good'
                  ELSE 'Invalide grade'
              END;
END;
```

Impliziter Ausdruck

Ausführungsteil
CASE-Anweisung

- CASE-Ausdruck prüft Bedingung und **führt Aktion aus**

```
DECLARE
  deptid NUMBER;
  deptname VARCHAR2(20);
  emps NUMBER;
  mngid NUMBER:= 108;
BEGIN
  CASE mngid
    WHEN 108 THEN
      SELECT department_id, department_name
             INTO deptid, deptname FROM departments
             WHERE manager_id=108;
      SELECT COUNT(*)
             INTO emps FROM employees
             WHERE department_id=deptid;
    WHEN 200 THEN
      ...
  END CASE;
...
END;
```

Ausführungsteil
Schleifenkonstrukte

LOOP-Schleife (Basisschleife)

```
[<< <Label> >>]
LOOP
  ...
  [EXIT [<Label>] [WHEN <Bedingung>];]
...
END LOOP [<Label>] ;
```

FOR-Schleife

```
FOR <Zaehler> IN [REVERSE] <StartAusdruck> .. <EndeAusdruck> LOOP
...
END LOOP;
```

WHILE-Schleife

```
WHILE <Bedingung> LOOP
...
END LOOP;
```

Ausführungsteil

Schleifenkonstrukte - Beispiele

```
DECLARE
  countryid  locations.country_id%TYPE := 'CA';
  loc_id     locations.location_id%TYPE;
  new_city   locations.city%TYPE := 'Montreal';
BEGIN
  SELECT MAX(location_id) INTO loc_id FROM locations
  WHERE country_id = countryid;
  FOR i IN 1..3 LOOP
    INSERT INTO locations(location_id, city, country_id)
    VALUES((loc_id + i), new_city, countryid);
  END LOOP;
END;
```

```
DECLARE
  …;
  counter     NUMBER := 1;
BEGIN
  …
  WHILE counter <= 3 LOOP
    INSERT INTO locations(location_id, city, country_id)
    VALUES((loc_id + counter), new_city, countryid);
    counter := counter +1;
  END LOOP;
END;
```

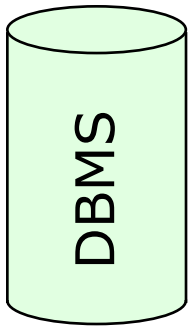
© 2025

Überblick

- Einführung und Motivation
- Deklarationsteil
- Ausführungsteil
- Cursorkonzept
- Gespeicherte Programme
- Pakete
- Ausnahmebehandlung
- Rechte

Cursorkonzept

Mengenorientierung versus Satzorientierung



Anwendung

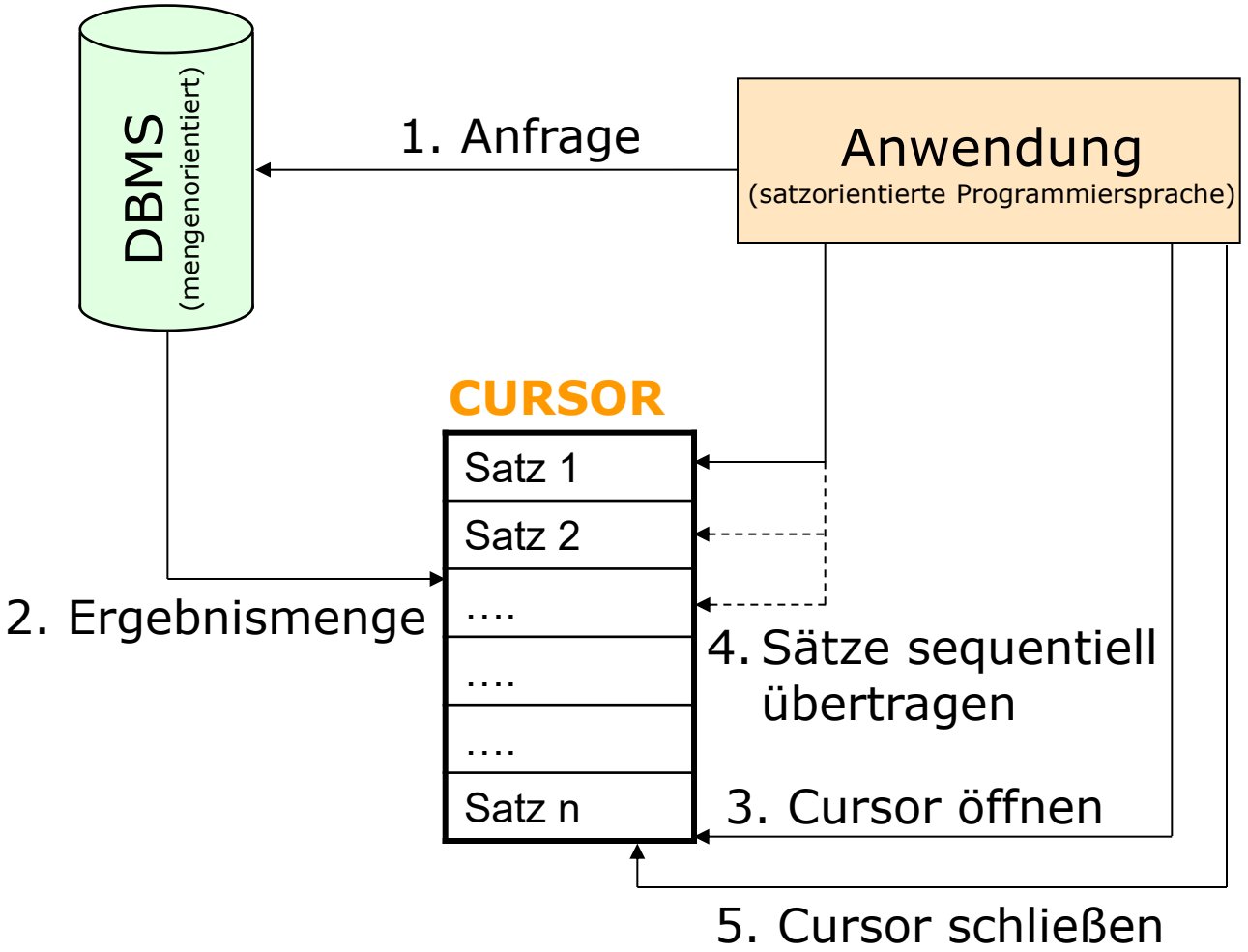
`SELECT`-Anweisung erzeugt eine Menge von Tupeln (= Relation).
⇨ **SQL**

In der Anwendung wird auf einzelne Datensätze (= Tupel) zugegriffen.
⇨ **Programmiersprache**

Übergang muss geschaffen werden.
⇨ CURSOR = Iterator

Cursor/Iterator ist einer Anfrage zugeordnet und stellt der Anwendung die Zeilen der Ergebnismenge einzeln (one tuple at a time) bereit.

Cursorkonzept
Überblick 1/2



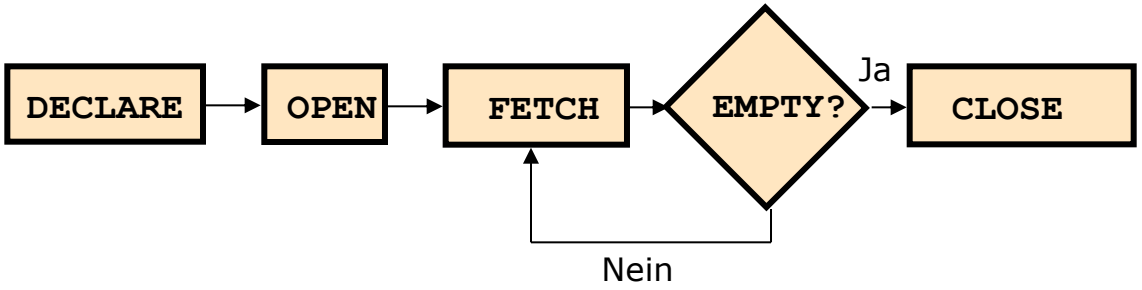
Cursorkonzept
Überblick 2/2

- **Cursor = Referenz auf privaten Arbeitsbereich** einer SQL-Anweisung („Private SQL Area“)

- Impliziter Cursor
  - für jede SQL-Anweisung, die nur eine einzige Zeile liefert
  - wird automatisch vom DB-Server erstellt und verwaltet
- Expliziter Cursor
  - für mengenorientierte Abfragen, die mehr als eine Zeile liefern können
  - ermöglicht **zeilenweise Abarbeitung** des Abfrageergebnisses
  - muss im Programm explizit deklariert werden

Cursorkonzept

Vorgangsweise bei expliziten Cursor



- **Deklarieren** eines Cursors
- **Öffnen** des Cursors
  - veranlasst Ausführung der SQL-Anweisung am Server ohne jedoch Daten in den Programmbereich zu transferieren
- **Übertragen** von Datensätzen aus der Datenbank in den Programmbereich
  - mit dem SQL-Befehl FETCH wird der jeweils nächste Datensatz in die spezifizierte Variable geladen
- **Schließen** des Cursors
  - definiertes Beenden der Leseoperation und Freigeben aller Ressourcen

© 2025

Cursorkonzept
Deklaration

- Deklaration von Name (max. 30 Zeichen), Eingabeparameter (optional) und `SELECT`-Anweisung (beliebig komplex)

Notation

```
CURSOR <Cursor> [(<Param> [{, <Param>}])]
IS <SFW-Block> [FOR UPDATE [OF <Attribute>]
[NOWAIT | WAIT n]];
```

Beispiel

```
CURSOR c1 (name_var VARCHAR2) IS   -- Datentypen ohne Größenangabe!
  SELECT mit_name, mit_geb, mit_abtnr
  FROM mitarbeiter
  WHERE mit_name LIKE name_var
  ORDER BY mit_name;
```

- **Default-Werte** für Eingabeparameter möglich

```
CURSOR c2 (low INTEGER := 0, high INTEGER := 99) IS ...;
```

- Generell keine Beschränkung für Anzahl der aktiven Cursor pro PL/SQL-Programm bzw. pro Session
  - aber über DB-Parameter `OPEN_CURSORS` definierbar

Cursorkonzept
Cursor-Attribute

- Zur Überprüfung der Cursor-Statusinformationen

Attribut	Wert
%FOUND	'wahr', wenn zuletzt ausgeführte FETCH-Anweisung Datensatz gefunden hat
%NOTFOUND	'wahr', wenn zuletzt ausgeführte FETCH- Anweisung keinen Datensatz gefunden hat
%ROWCOUNT	Anzahl bisher gelesener Datensätze
%ISOPEN	'wahr', wenn der Cursor geöffnet ist

- Bei **expliziten Cursorn** Namen mit Attribute ergänzen

```
IF NOT c1%ISOPEN THEN
  OPEN c1('Huber');
END IF;
...
FETCH c1 INTO ...
IF c1%FOUND THEN ...
```

- Attribute auch bei **impliziten Cursorn** verwendbar
  - impliziter Cursorn sind in SQL verfügbar

```
UPDATE mitarbeiter SET mit_gehalt = mit_gehalt * 1.05
WHERE mit_abtnr = 40;
anzahl := SQL%ROWCOUNT;
```

Impliziter Cursor

Cursorkonzept

Cursor-Verarbeitung 1/3

```
DECLARE
  ...
  CURSOR C1 IS   SELECT * FROM mitarbeiter
                 ORDER BY mit_name
                 FOR UPDATE;
  c1_rec C1%ROWTYPE;

BEGIN
  ...
  OPEN C1;
  LOOP
    FETCH C1 INTO c1_rec;
    EXIT WHEN C1%NOTFOUND;
    /*Wenn im Cursor keine Datensätze mehr    */
    /*vorhanden sind, verlasse die Schleife   */
    ...
    UPDATE mitarbeiter SET comm = 70 WHERE CURRENT OF C1;
  END LOOP;
  ...
  anzahl := C1%ROWCOUNT;
  CLOSE C1;
  ...
END;
```

FOR UPDATE-Klausel
Explizite Sperren für die Dauer der Transaktion anfordern

WHERE CURRENT OF-Klausel
referenziert aktuelle Zeile

Cursorkonzept
Cursor-Verarbeitung 2/3

- Cursor-Parameterwerte beim Öffnen übergeben.
- Cursor kann mehrmals geöffnet werden mit unterschiedlichen Parametern.

```
DECLARE
  ...
  CURSOR emp_cursor(deptno NUMBER) IS
      SELECT employee_id, last_name
      FROM employees
      WHERE department_id = deptno;
  dept_id   NUMBER;
  lname     VARCHAR2(15);

BEGIN
  ...
  OPEN emp_cursor(10);
    ...
  CLOSE emp_cursor;
  OPEN emp_cursor(20);
    ...
END;
```

parametrisierter Cursor

erneutes OPEN erfordert vorhergehendes CLOSE

Cursorkonzept
Cursor-Verarbeitung 3/3

- **Cursor-FOR-Schleife** (syntaktischer Zucker)
  - implizite Durchführung von
    - Erstellung eines Records laut Deklaration
    - Öffnen (mit Paramenter)
    - Übertragen der Datensätze (DB → Record)
    - Schließen des Cursors und Beenden der Schleife
      - » auch bei EXIT oder Exceptions

```
DECLARE
  CURSOR emp_cursor IS
    SELECT employee_id, last_name FROM employees
    WHERE department_id =30;
BEGIN
   FOR emp_record IN emp_cursor  -- emp_record wird implizit deklariert
    LOOP
     DBMS_OUTPUT.PUT_LINE(emp_record.employee_id
     ||' ' ||emp_record.last_name);
    END LOOP;
END;
```

Review-Fragen

- Anbindungsmöglichkeiten von prozeduralen Programmen an deklarative Datenbanken?
- Vor- und Nachteile von PSM bzw. PL/SQL?
- Aufbau einer Stored-Procedure?
- spezielle Typkonstruktoren?
- neue Datentypen?
- zulässige/unzulässige Anweisungstypen?
  - Kontrollfluss-Anweisungen?
- Cursor?
  - Lebenszyklus?
  - Parameter?
  - %Properties?
  - impliziter Cursor?
  - Schleifen?

Überblick

- Einführung und Motivation
- Deklarationsteil
- Ausführungsteil
- Cursorkonzept
- Gespeicherte Programme
- Pakete
- Ausnahmebehandlung
- Rechte

Gespeicherte Programme
Prozeduren - Funktionen

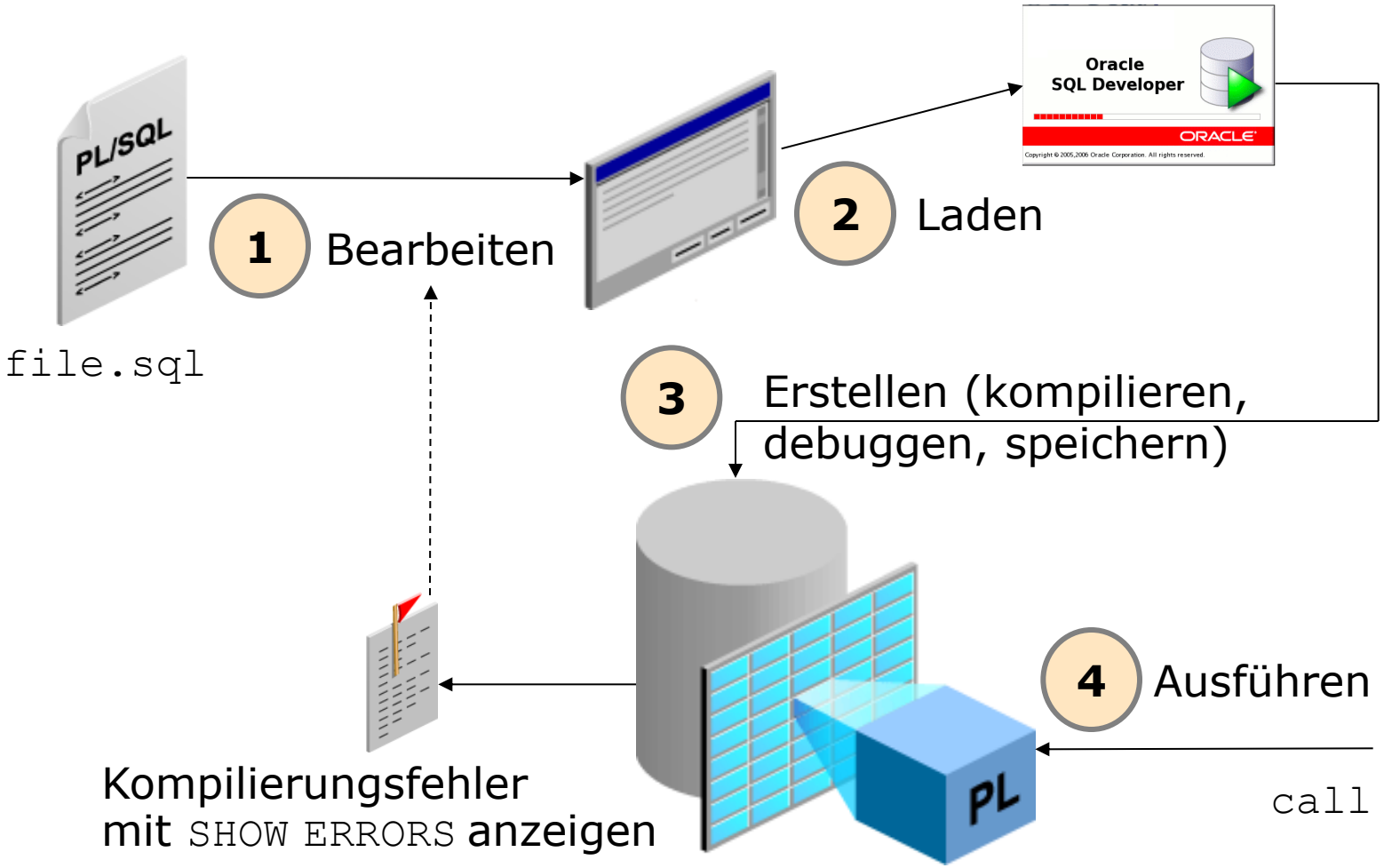
- Prozeduren und Funktionen
  - Speicherung in kompilierter Form in der Datenbank
  - bei Aufruf Ausführung im DB-Server ⇨ bessere Performanz

Anonyme Blöcke	**Gespeicherte Prozeduren/Funktionen**
Unbenannte PL/SQL-Blöcke	Benannte PL/SQL-Blöcke
Jedes Mal kompiliert	Nur einmal kompiliert
Nicht in der Datenbank gespeichert	In der Datenbank gespeichert
Können nicht von anderen Anwendungen aufgerufen werden	Werden benannt und können daher von anderen Anwendungen aufgerufen werden
Geben keine Werte zurück	Als Funktionen bezeichnete Unterprogramme müssen Werte zurückgeben
Können keine Parameter annehmen	Können Parameter annehmen

© 2025

Gespeicherte Programme

Prozeduren – Funktionen entwickeln



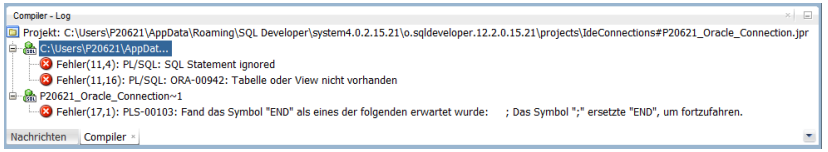
Gespeicherte Programme
Prozeduren – Funktionen definieren

- **Definition** einer gespeicherten Prozedur/Funktion durch DDL-Anweisungen

Notation

```
CREATE OR REPLACE PROCEDURE / FUNCTION
```

  - **PL/SQL-Compiler** prüft Syntax und Semantik
  - **SQL-Anweisungen werden geparst** (Prüfen von Objektnamen, Spaltennamen, Zugriffsberechtigungen)
  - **Sourcecode** u. **übersetztes PL/SQL-Programm** werden im Data Dictionary (Source in Tabelle `source$` - View `user_source`) gespeichert (auch bei fehlerhafter Kompilierung)
  - Fehler werden im Data Dictionary in Tabelle `error$` (View `user_errors`) protokolliert
  - Ermittlung der Fehlerursache
    - mittels SQL: `SELECT * FROM USER_ERRORS WHERE name=<Funk/ProzName>`
    - mittels Werkzeugunterstützung:



Gespeicherte Programme
Prozeduren – Syntax

Notation

```
CREATE [OR REPLACE] PROCEDURE <ProzName>
  [(<Param> [IN | OUT | IN OUT] <Datentyp>                    Block Header
  [:=|DEFAULT <Ausdruck>], ...)]      -- Datentypen ohne Größenangabe!
AS | IS
  [<lokale Deklarationen>]                                    Deklarationsblock
BEGIN
  <PL/SQL-Code>                                               Ausführungsblock
[EXCEPTION
  <exception handler> [{<exception handler>}]]                Fehlerbehandlungsblock
END [ProzName];
```

- Parameterübergabe
  - `IN`-Parameter (Default): übergibt Wert für die Verarbeitung an die Prozedur
  - `OUT`-Parameter: gibt dem Aufrufer einen Wert zurück
  - `IN OUT`-Parameter: stellt einen Eingabewert bereit, der als modifizierter Wert zurückgegeben werden kann

Gespeicherte Programme
Prozeduren

Beispiel

```
CREATE OR REPLACE PROCEDURE raiseSalary (
      emp_id IN employees.employee_id%TYPE,
      percent IN NUMBER DEFAULT 0)
AS
      currentSalary employees.salary%TYPE;
BEGIN
  SELECT  salary INTO currentSalary FROM employees
  WHERE   employee_id = emp_id;

  IF currentSalary IS NULL THEN
     INSERT INTO empAudit VALUES (emp_id, 'Salary is null');
  ELSE
    UPDATE employees SET salary = salary * (1 + percent/100)
    WHERE employee_id= emp_id;
  END IF;
END raiseSalary;
```

Formalparameter

```
Aufruf in PL/SQL: raiseSalary(100, 3);
Aufruf in SQL-Umgebung: EXEC[UTE] raiseSalary(100, 3);
```

Aktualparameter

Gespeicherte Programme
Funktionen – Syntax

Notation

```
CREATE [OR REPLACE] FUNCTION <FunkName>
  [(<Param> ...)]                                          Block Header
  RETURN <Datentyp>    -- Datentypen ohne Größenangabe!
AS | IS
  [<lokale Deklarationen>]                                 Deklarationsblock
BEGIN
  <PL/SQL-Code>        -- RETURN <Ausdruck>;               Ausführungsblock
  [EXCEPTION
    <exception handler> [{<exception handler>}]]           Fehlerbehandlungsblock
END [FunkName];
```

- Funktion darf nicht ohne Rückgabewert beendet werden
- Funktion kann auch in SQL-Abfragen verwendet werden

Gespeicherte Programme
Funktionen

- Funktionen können in SQL-Anweisungen verwendet werden

Beispiel

```
CREATE OR REPLACE FUNCTION tax(value IN NUMBER)
 RETURN NUMBER IS
BEGIN
  RETURN (value * 0.08);
END tax;
```

```
SELECT employee_id, last_name, salary, tax(salary)
FROM  employees
WHERE department_id = 100;
```

Gespeicherte Programme

Funktionen – Einschränkungen innerhalb von SQL/DML

- Einschränkungen von Funktionen innerhalb von Abfragen um **Seiteneffekte** (*Mutating Function*) zu vermeiden
  - `SELECT` kann kein DML anstoßen
  - `UPDATE`/`DELETE` können kein DML in derselben Tabelle anstoßen
  - SQL-Anweisungen können Transaktionen nicht beenden (d. h. keine `COMMIT`- oder `ROLLBACK`-Operationen ausführen, bzw. auch keine DDL-Anweisungen)

- Gilt auch indirekt: Aufrufe von Unterprogrammen, die gegen diese Einschränkungen verstoßen, sind in der Funktion ebenfalls nicht zulässig.

Gespeicherte Programme

Prozeduren und Funktionen – entfernen/rekompilieren

- Prozedur/Funktion löschen
  - vergebene Privilegien der Prozedur/Funktion werden entzogen

Notation

```
DROP PROCEDURE <ProzName>
DROP FUNCTION <FunkName>
```

- **CREATE OR REPLACE**
  - vergebene Privilegien der Prozedur/Funktion bleiben erhalten

- Prozedur/Funktion neu übersetzen

Notation

```
ALTER PROCEDURE <ProzName> COMPILE;
ALTER FUNCTION <FunkName> COMPILE;
```

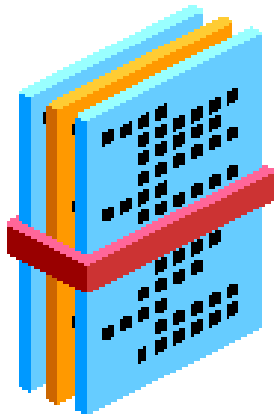
© 2025

Überblick

- Einführung und Motivation
- Deklarationsteil
- Ausführungsteil
- Cursorkonzept
- Gespeicherte Programme
- Pakete
- Ausnahmebehandlung
- Rechte

Pakete
Überblick

- Pakete
  - kapseln Prozeduren, Funktionen, Typen, Variablen, Ausnahmen und Zugriffsprivilegien zu einem **Modul** bzw. zu einer **Bibliothek**/Einheit zusammen
  - bestehen aus zwei Teilen
    - Spezifikationsteil (Schnittstelle)
    - Rumpf (falls erforderlich) (Implementierung)
  - werden in der Datenbank gespeichert



Pakete
Komponenten

Paketspezifikation



Variable

Deklaration Prozedur A;

öffentlich

Paketrumpf



Variable

Definition Prozedur B …

Definition Prozedur A

Variable

```
BEGIN
…
END;
```

privat

Pakete
Spezifikation/Rumpf

- Definition von **Paketspezifikation** und **-rumpfes**

gleicher Name

Paketspezifikation

```
CREATE [OR REPLACE] PACKAGE <Paketname> AS|IS
  <globale Deklarationen: Datentypen, Variablen, ... >
  <Prozedur- & Funktionsdeklarationen>
END [<Paketname>];
```

Paketrumpf

```
CREATE [OR REPLACE] PACKAGE BODY <Paketname> AS|IS
  <lokale Deklarationen: Datentypen, Variablen, ... >
  <Prozedur- & Funktionsrümpfe>
[BEGIN
  <Initialisierungsanweisungen>]
END [<Paketname>];
```

← optionaler Package-Initialisierungsblock

Pakete
Beispiel - Spezifikation

Beispiel

```
CREATE OR REPLACE PACKAGE emp_actions_pkg AS

  TYPE EmpRecTyp IS RECORD
                  (emp_id employees.employee_id%TYPE,
                   sal employees.salary%TYPE);

  CURSOR desc_salary RETURN EmpRecTyp;

  PROCEDURE hire_employee (f_name VARCHAR2, l_name VARCHAR2, mail
         VARCHAR2, job VARCHAR2, mgr NUMBER, sal NUMBER, comm NUMBER,
         deptno NUMBER);

  PROCEDURE fire_employee (emp_id NUMBER);

END emp_actions_pkg;
```

Pakete
Beispiel - Rumpf

Beispiel

```
CREATE OR REPLACE PACKAGE BODY emp_actions_pkg AS
  CURSOR desc_salary RETURN EmpRecTyp IS
    SELECT employee_id, salary FROM employees ORDER BY salary DESC;

  PROCEDURE hire_employee (f_name VARCHAR2, l_name VARCHAR2, mail
         VARCHAR2, job VARCHAR2, mgr NUMBER, sal NUMBER, comm NUMBER,
         deptno NUMBER) IS
  BEGIN
    INSERT INTO employees (employee_id, first_name, last_name, email,
      hire_date, job_id, salary, commission_pct, manager_id,
      department_id)
      VALUES (employees_seq.NEXTVAL, f_name, l_name, mail, SYSDATE,
                  job, sal, comm, mgr, deptno);
  END hire_employee;

  PROCEDURE fire_employee (emp_id NUMBER) IS
  BEGIN
    DELETE FROM employees WHERE employee_id = emp_id;
  END fire_employee;

END emp_actions_pkg;
```

Pakete
Anwendung

- Aufruf von Standalone-Prozeduren und -Funktionen durch
  <ersteller/schema>.<prozName>(<Aktualparameter>)

```
altmann.raise_salary(huber_id, 100);
```

- Aufruf von Paket-Prozeduren und -Funktionen durch
  <ersteller/schema>.<paketName>.<prozName> (<Aktualparameter>)

```
altmann.emp_actions_pkg.hire_employee('King', ...);
```

  - Verwendung von Synonymen zur Vereinfachung

```
CREATE SYNONYM e_actions FOR altmann.emp_actions_pkg;
```

- Zugriff auf Paket-Typen, -Variablen, -Cursor

```
OPEN altmann.emp_actions_pkg.desc_salary;
OPEN e_actions.desc_salary;
...
```

Pakete
Überladen (Overloading)

- Package-Prozeduren/Funktionen können gleich benannt werden.
- durch unterschiedliche Anzahl oder unterschiedliche Datentypen der Parameter unterscheidbar

Beispiel

```
CREATE PACKAGE xyz
IS
     -- zulässig, da erste Deklaration
     PROCEDURE a (p1 NUMBER);
     -- zulässig, da unterschiedliche Anzahl an Parametern
     PROCEDURE a (p1 NUMBER, p2 NUMBER);
     -- zulässig, da unterschiedliche Datentypen
     PROCEDURE a (p1 DATE);
     -- a (p3 NUMBER) NICHT zulässig, da weder unterschiedliche
     -- Anzahl noch unterschiedliche Datentypen zur 1. Definition
     PROCEDURE a (p3 NUMBER);
END;
```

- Achtung: Fehler erst beim Aufruf (zur Laufzeit!)

```
PLS-00307: too many declarations of 'P' match this call
```

Pakete

Pakete entfernen/rekomplieren

- Paket-Spezifikation und -Rumpf entfernen

Notation

```
DROP PACKAGE <PackageName>;
```

- Nur Paket-Rumpf entfernen

Notation

```
DROP PACKAGE BODY <PackageName>;
```

- Paket mit **ALTER**-Anweisung rekompilieren

Notation

```
ALTER PACKAGE <PackageName> COMPILE SPECIFICATION;
ALTER PACKAGE <PackageName> COMPILE BODY;
ALTER PACKAGE <PackageName> COMPILE; -- both
```

© 2025

Pakete
Oracle-Bibliothek

- DBMS_STANDARD
  - RAISE_APPLICATION_ERROR
- DBMS_OUTPUT
  - DBMS_OUTPUT.PUT_LINE(Text)
- DBMS_SQL
  - Dynamisches SQL
- DBMS_TRANSACTION
- UTL_FILE
  - Dateien schreiben und lesen
- DBMS_SESSION
- DBMS_UTILITY
- HTML
- ...

© 2025

Überblick

- Einführung und Motivation
- Deklarationsteil
- Ausführungsteil
- Cursorkonzept
- Gespeicherte Programme
- Pakete
- Ausnahmebehandlung
- Rechte

Ausnahmebehandlung
Beispiel 1/2

Beispiel

```
. . .
DECLARE
  lname VARCHAR2(15);
BEGIN
    SELECT last_name INTO lname FROM employees WHERE
    first_name = 'Peter';
    . . .
END;
```

Fehler zur Laufzeit, wenn mehrere Zeilen selektiert werden!

```
ERROR: ERROR at line 1:
ORA-01422: exact fetch returns more than requested number of rows
. . .
```

Programmabruch

Fehler, die zur Laufzeit auftreten, werden als **Ausnahmen** (**Exceptions**) bezeichnet

Ausnahmebehandlung
Beispiel 2/2

Beispiel

```
. . .
DECLARE
  lname VARCHAR2(15);
BEGIN
    SELECT last_name INTO lname FROM employees WHERE
    first_name = 'Peter';
    . . . ;
EXCEPTION
    WHEN TOO_MANY_ROWS THEN
      DBMS_OUTPUT.PUT_LINE ('Your SELECT-Statement retrieved
      multiple rows. Consider using a cursor.');
END;
```

Ausnahme wird ausgelöst

Ausnahme wird behandelt

```
Your SELECT-Statement retrieved multiple rows. Consider using a cursor.
PL/SQL-Prozedur wurde erfolgreich abgeschlossen.
```

- Anweisungen im Exception-Bereich werden ausgeführt und
- Block wird ordnungsgemäß abgeschlossen

Ausnahmebehandlung

Auslösen und Behandeln

- Eine **Ausnahme** ist ein **Fehler** in PL/SQL, der bei der Ausführung ausgelöst wird.

- Eine **Ausnahme** kann wie folgt **ausgelöst** werden:
  - Implizit vom Oracle-Server (→ Oracle-Fehler tritt auf)
  - Explizit vom Programm (→ Fehler der Programmlogik)

- Eine **Ausnahme** kann wie folgt **behandelt** werden:
  - Durch Abfangen mit einem **Exception Handler**
  - Durch Propagieren an die aufrufende Umgebung (z.B. SQL*Plus, PL/SQL-Programm, umschließender Block)

© 2025

Ausnahmebehandlung

Arten von Ausnahmen

- Systemdefinierte Ausnahmen (ca. 20 benannte Exceptions)
  - Keine Deklaration notwendig
  - Aktivierung erfolgt automatisch (durch Oracle-Server)
- Benutzerdefinierte Ausnahmen
  - `TOO_MANY_CURSORS` **`EXCEPTION;`**
    `MY_EXCEPTION` **`EXCEPTION;`**
  - Aktivierung durch **`RAISE`** `<Ausnahme>;` im Ausführungsteil
    `RAISE` `MY_EXCEPTION;`
  - Abbildung von Oracle-Fehlernummern auf Exceptions
    - z.B. für Fehlernummern ohne Namen
    - **`PRAGMA`** `EXCEPTION_INIT (MY_EXCEPTION, -20001);`
    - können dann im Exception-Teil über Namen angesprochen werden

Ausnahmebehandlung

Systemdefinierte Ausnahmen (Auswahl)

Ausnahmen/Fehlername	Fehlernummer von Oracle-Server	Beschreibung
CURSOR_ALREADY_OPEN	ORA-06511	Es wurde versucht, einen bereits geöffneten Cursor zu öffnen.
DUP_VAL_ON_INDEX	ORA-00001	Es wurde versucht, einen Wert mehrfach einzufügen.
INVALID_CURSOR	ORA-01001	Ein ungültiger Cursor-Vorgang ist aufgetreten.
INVALID_NUMBER	ORA-01722	Die Konvertierung einer Zeichenfolge in eine Zahl ist misslungen
LOGIN_DENIED	ORA-01017	Anmeldung beim Oracle-Server gescheitert.
NO_DATA_FOUND	**ORA-01403**	**SELECT zur Rückgabe einer Zeile gab keine Daten zurück.**
NOT_LOGGED_ON	ORA-01012	Ein PL/SQL-Programm hat einen Datenbankaufruf abgesetzt, ohne mit dem Oracle-Server verbunden zu sein.
PROGRAM_ERROR	ORA-06501	Internes PL/SQL-Problem.
STORAGE_ERROR	ORA-06500	Zu wenig Speicher für PL/SQL oder Speicher beschädigt.
TIMEOUT_ON_RESOURCE	ORA-00051	Zeitüberschreitung, während der Oracle-Server auf eine Ressource wartete.
TOO_MANY_ROWS	**ORA-01422**	**SELECT zur Rückgabe einer Zeile gab mehr als eine Zeile zurück.**
VALUE_ERROR	ORA-06502	Arithmetischer, Konvertierungs-, Truncation- oder Größenbeschränkungsfehler
ZERO_DIVIDE	**ORA-01476**	**Versuchte Division durch null.**

Vollständige Liste: PL/SQL User's Guide and Reference
Vordefinierte Exceptions im STANDARD-Package definiert.

© 2025

Ausnahmebehandlung

Nicht vordefinierte Ausnahmen

Oracle-Server-Fehlernummer -01400 abfangen
(= kann NULL nicht einfügen)

1. Namen der Ausnahme deklarieren

2. Standardfehlernummer des Oracle-Servers zuordnen

Beispiel

```
. . .
DECLARE
  INSERT_EXCEPTION EXCEPTION;
  PRAGMA EXCEPTION_INIT
  (INSERT_EXCEPTION, -01400);
BEGIN
  INSERT INTO departments (department_id, department_name)
  VALUES (280, NULL);
  . . .
EXCEPTION
  WHEN INSERT_EXCEPTION THEN
    . . .
  WHEN OTHERS THEN
    . . .
END;
```

3. Deklarierte Ausnahme referenzieren

Optionale Klausel zur Behandlung von Ausnahmen, die alle Ausnahmen abfängt, die nicht explizit behandelt wurden.

ORA-01400
cannot insert NULL into (*string*)

Cause An attempt was made to insert NULL into previously listed objects.

Action These objects cannot accept NULL values. Reservable columns cannot accept NULL values.

Ausnahmebehandlung

Benutzerdefinierte Ausnahmen

(dafür existieren keine vordefinierten Oracle-Fehlernummern)

Beispiel

```
. . .
DECLARE
  INVALID_DEPARTMENT EXCEPTION;
  name VARCHAR2(20):='&name';
  deptno NUMBER :=&deptno;
BEGIN
  UPDATE  departments
  SET     department_name = name
  WHERE   department_id = deptno;
  IF SQL%NOTFOUND THEN
    RAISE INVALID_DEPARTMENT;
  END IF;
EXCEPTION
  WHEN INVALID_DEPARTMENT THEN
    DBMS_OUTPUT.PUT_LINE('No such department id.');
END;
```

1. Namen der Ausnahme deklarieren

2. Auslösen der Ausnahme

3. Deklarierte Ausnahme referenzieren

ANMERKUNG: Nur SELECT kann automatisch eine Exception auslösen - UPDATE, DELETE und INSERT nicht!
Workaround → Status vom impliziten Cursor abfragen, da UPDATE keine Exception auslösen kann!

Ausnahmebehandlung

Exception Handler

- Ausnahmebehandlungsteil wird durch Schlüsselwort **EXCEPTION** eingeleitet
- Definition von **Exception-Handlern**, die eine oder mehrere Fehlersituationen bearbeiten

```
WHEN <Ausnahme> {OR <Ausnahme>} THEN <PL/SQL-Anweisungen>
```

- optional OTHERS für andere Fehlersituationen

```
WHEN OTHERS THEN <PL/SQL-Anweisungen>
```

- unbehandelte Ausnahmen werden **an den nächsten übergeordneten PL/SQL-Block propagiert**
- Nach Ausführung des Exception-Blocks wird der PL/SQL-Block verlassen und mit der **nächsten Anweisung des übergeordneten Blocks** fortgefahren

Ausnahmebehandlung

Ausnahmen propagieren 1/2

Beispiel

```
. . .
BEGIN
  . . .
        BEGIN
          IF x = 1 THEN
            RAISE a;
          ELSIF x = 2 THEN
            RAISE b;
          ELSE
            RAISE c;
          END IF;
        ...
        EXCEPTION
          WHEN a THEN
          ...
        END;
  . . .
EXCEPTION
  WHEN b THEN
    . . .
END;
```

kein geeigneter Exception-Handler für Ausnahme b im innersten Block

Propagierung der Ausnahme b in den umschließenden Block

Nach Ausnahmebehandlung Kontrolle an Host-Umgebung oder Propagierung an den umschließenden Block

Ausnahmebehandlung

Ausnahmen propagieren 2/2

Beispiel

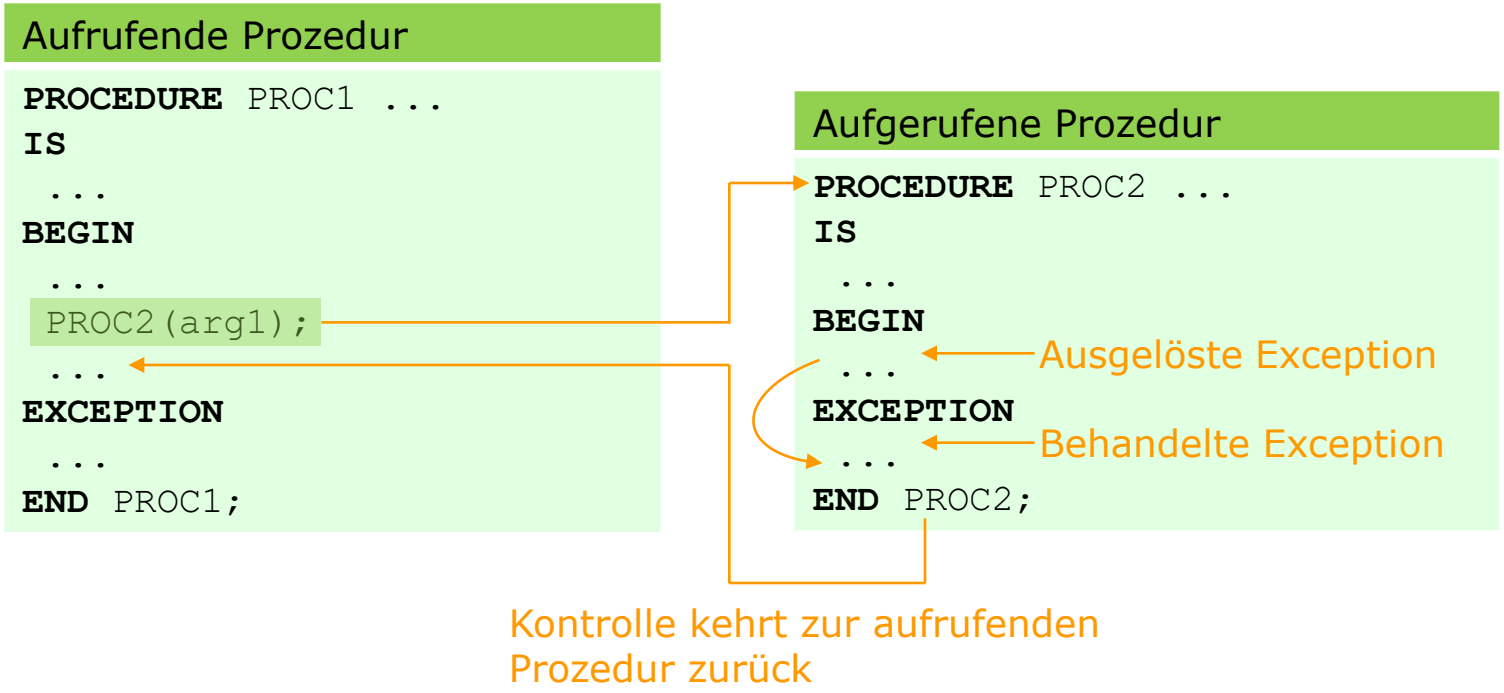
```
DECLARE
   . . .
   NO_ROWS             EXCEPTION;
   INTEGRITY_ERROR     EXCEPTION;
   PRAGMA EXCEPTION_INIT (INTEGRITY_ERROR, -2292);
BEGIN
   FOR c_record IN emp_cursor LOOP
      BEGIN
       SELECT ...
       UPDATE ...
       IF SQL%NOTFOUND THEN
        RAISE NO_ROWS;
       END IF;
      END;
   END LOOP;
EXCEPTION
   WHEN INTEGRITY_ERROR THEN ...
   WHEN NO_ROWS THEN ...
END;
```

Oracle-Fehler -2292:
Verletzung der referentiellen Integrität

NO_DATA_FOUND

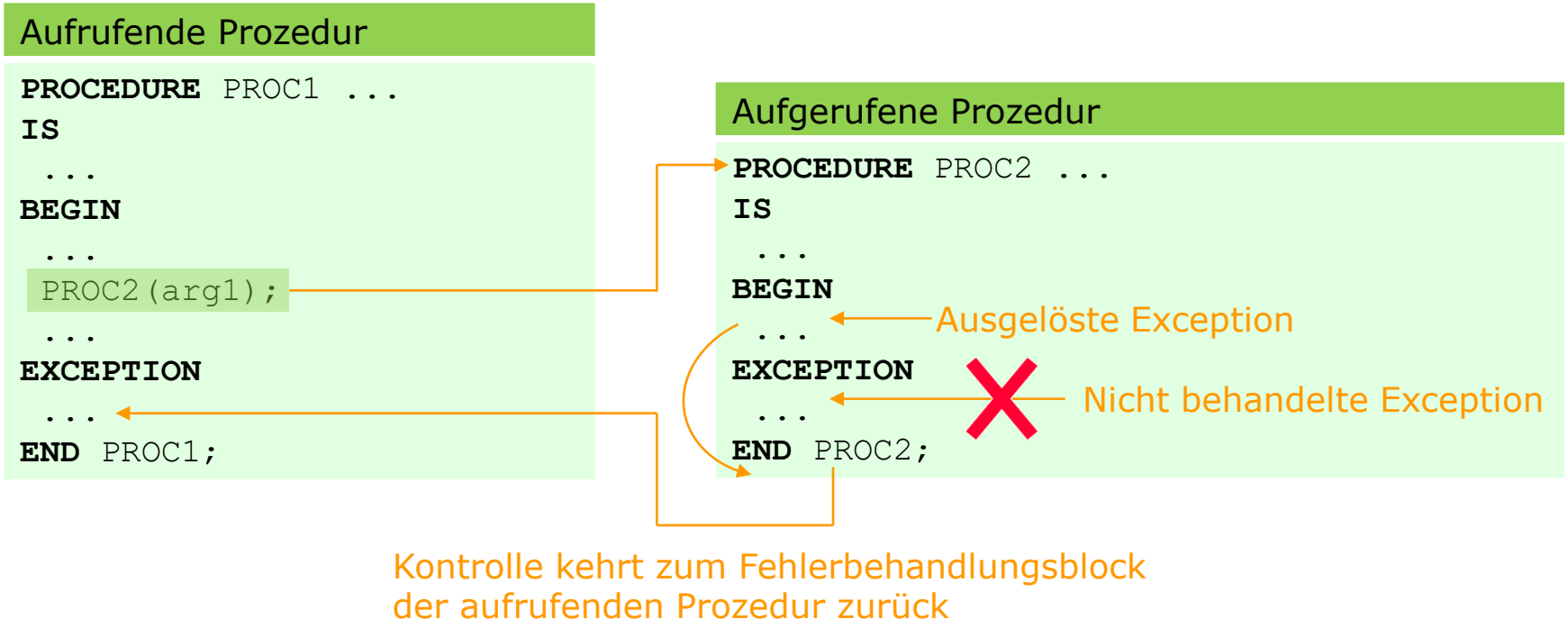
Ausnahmebehandlung

Prozeduren – Behandelte Ausnahmen



Ausnahmebehandlung

Prozeduren – Nicht behandelte Ausnahmen



Ausnahmen & Transaktionen

- bleibt Ausnahme unbehandelt werden Änderungen innerhalb der Prozedur zurückgenommen

Prozedur mit Fehler beim zweiten Aufruf

```
CREATE OR REPLACE PROCEDURE fail_and_rollback AS
  test_count INTEGER;
  test_id INTEGER;
BEGIN
  SELECT COUNT(*) INTO test_count FROM TEST;
  dbms_output.put_line('test has ' || test_count || ' rows');
  INSERT INTO TEST VALUES (test_count, 'value');
  SELECT ID INTO test_id FROM TEST;
END;
CALL fail_and_rollback();
CALL fail_and_rollback();
CALL fail_and_rollback();
```

```
test has 0 rows
test has 1 rows
SQL-Fehler: ORA-01422: exact fetch returns more than requested number of rows
test has 1 rows
SQL-Fehler: ORA-01422: exact fetch returns more than requested number of rows
```

Ausnahmebehandlung

SQLCODE und SQLERRM 1/3

- Innerhalb des Ausnahmebehandlungsteils kann über die Fehlerfunktionen **SQLCODE** und **SQLERRM** auf den Oracle-Fehlercode und den zugehörigen Fehlertext zurückgegriffen werden
  - **SQLCODE**: gibt den numerischen Wert für den Fehlercode zurück

SQLCODE-Wert	Beschreibung
0	Keine Ausnahme aufgetreten
1	Benutzerdefinierte Ausnahme (ohne Zuordnung zu Oracle-Fehler)
+100	Ausnahme NO_DATA_FOUND (entspricht internem Oracle-Code ORA-01403)
< 0	Eine andere Oracle-Server-Fehlernummer

  - **SQLERRM**: gibt die zu SQLCODE passende Meldung zurück

Ausnahmebehandlung

SQLCODE und SQLERRM 2/3

Beispiel

```
BEGIN
  INSERT INTO TEST VALUES (1, '2');
  INSERT INTO TEST VALUES (1, '2');
EXCEPTION
  WHEN others THEN
    dbms_output.put_line('code ' || SQLCODE|| ': ' || SQLERRM);
END;
```

```
PL/SQL-Prozedur erfolgreich abgeschlossen.
code -1 ORA-00001: unique constraint (SYS_C0028709) violated
```

SQLCODE **SQLERRM**

Ausnahmebehandlung
SQLCODE und SQLERRM 3/3

- **SQLCODE** und **SQLERRM** sind nicht direkt in SQL-Anweisungen verwendbar (Zuweisung zu lokaler Variable notwendig).

Beispiel

```
DECLARE
  error_code      NUMBER;
  error_message   VARCHAR2(100);
  . . .
BEGIN
  . . .
EXCEPTION
  . . .
  WHEN OTHERS THEN
    error_code := SQLCODE;
    error_message := SUBSTR(SQLERRM,1,100);
    INSERT INTO   errors(e_user, e_date, error_code, error_message)
    VALUES(USER, SYSDATE, error_code, error_message);
  . . .
```

Überblick

- Einführung und Motivation
- Deklarationsteil
- Ausführungsteil
- Cursorkonzept
- Gespeicherte Programme
- Pakete
- Ausnahmebehandlung
- Rechte

Rechtemodell

- Rechte des **Eigentümers**
  - Programme werden mit den Privilegien des erstellenden Benutzers ausgeführt.
  - Benutzer benötigen keine Privilegien für zu Grunde liegende Objekte, auf die die Prozedur zugreift. Benutzer benötigen lediglich ein Privileg zum Ausführen einer Prozedur.
- Rechte des **ausführenden Benutzers**
  - Programme werden mit den Privilegien des aufrufenden Benutzers ausgeführt.
  - Benutzer benötigen Privilegien für zu Grunde liegende Objekte, auf die die Prozedur zugreift.

© 2025

Rechte des ausführenden Benutzers

- **AUTHID**-Klausel legt fest
  - mit welchen Rechten eine Prozedur oder Paket aufgerufen wird
  - wie Schemaobjekte ohne Schema-Angabe aufgelöst werden

```
CREATE OR REPLACE PROCEDURE raiseSalary
      (   ... )
      AUTHID CURRENT_USER
AS
  ...
BEGIN
  SELECT salary INTO currentSalary FROM <owner>.employees
  WHERE employee_id = emp_id;
  ...
END raiseSalary;
```

Rechte des ausführenden Benutzers sind auch für die in der Prozedur verwendeten Schemaobjekte relevant.

z.B.

d.h.

Neben dem `EXECUTE`-Recht benötigt der ausführende Benutzer auch das `SELECT`-Recht auf die Tabelle `<owner>.employees`

- Standardwert: **AUTHID_DEFINER**

Zusammenfassung

Prozedurale SQL-Erweiterung …

- … vereint die Vorteile der deklarativen Abfragesprache SQL mit denen einer prozeduralen Sprache wie bspw.
  - Kontrollfluss
  - Blockstruktur
  - Prozedur, Funktion, Paket
  - Fehlerbehandlung
- … ermöglicht das Speichern und Ausführen von Programmen auf dem Datenbank-Server
- … unterstützt leistungsstarke und sichere SQL-Abfragen
- … ist eine proprietäre Sprache
- … bildet die Grundlage für die Umsetzung von
  - prozeduralen Integritätsbedingungen ⇨ Trigger
  - objektrelationalen Konzepten

© 2025

Review-Fragen

- PL/SQL in SQL?
- Mutating Function?
- Aufbau von Paketen?
- globale Variablen?
- Überladen von Funktionen?
- Ausnahmen?
  - Oracle Fehler als Ausnahme
  - Ausnahmen und Transaktionen?
- Rechte von Funktionen?

© 2025

Anhang: Dynamisches SQL

- SQL-Anweisung heißt **dynamisch**, wenn sie zur Laufzeit erstellt und dann auch ausgeführt wird.
  - Einzige Möglichkeit, DDL-Anweisungen in PL/SQL zu verarbeiten
- Früher: Build-in-Package DBMS_SQL
- Seit ORACLE 8 : Native Dynamic SQL (NDS)
  - Einfache Handhabung

```
EXECUTE IMMEDIATE 'Zeichenkette' ;

EXECUTE IMMEDIATE 'CREATE TABLE Test (...)';
EXECUTE IMMEDIATE 'ALTER TABLE Test DISABLE ALL CONSTRAINTS';

-- SELECT – Anweisungen erst zur Laufzeit erstellen
```

Anhang: Debug-Unterstützung

- Vordefiniertes PL/SQL-Paket `DBMS_OUTPUT` stellt Debug-Prozeduren zur Verfügung
  - `PUT_LINE(var)` schreibt Wert beliebiger Variable in neue Zeile
- Aktivierung der Ausgabe in (i)SQL*Plus mit `SET SERVEROUTPUT ON`
- in SQL-Developer
  - Ansicht → DBMS-Ausgabe

